Timetable Management System Software Requirements Specification (SRS)

Version 1.0
Date: 24.10.2026

Orhan Yalın Öztürk

Kürşad Şeker

Instructor: Dr. Buse Esirik

Fall 2026/2027

1 Introduction..... 4
1.1 Purpose..... 4
1.2 Scope..... 4
1.3 Definitions, Acronyms, and Abbreviations..... 5
1.4 References..... 5
1.5 Overview..... 6
2 General Description..... 6
2.1 Product Perspective..... 6
2.2 Product Functions..... 7
2.2.1 Data Management & Parsing..... 7
2.2.2 Automatic Timetable Generation..... 7
2.2.3 Timetable Viewing & Presentation..... 7
2.2.4 Manual Modification & Conflict Validation..... 7
2.2.5 Unscheduled Session & Classroom Request Management..... 7
2.2.6 Export & Output Persistence..... 7
2.3 User Characteristics..... 8
2.3.1 Department Course Timetable Coordinator..... 8
2.4 General Constraints..... 8
2.4.1 Programming Language & Platform Environment..... 8
2.4.2 Execution Paradigm & User Interface Constraints..... 8
2.4.3 Algorithmic & Architectural Scope Limits..... 8
2.4.4 Scheduling & Academic Boundary Constraints..... 8
2.4.5 Hard Constraint Inviolability..... 8
2.4.6 Soft Constraint Flexibility & Warning Notification..... 9
2.5 Assumptions and Dependencies..... 9
2.5.1 Runtime & Execution Dependencies..... 9
2.5.2 Input File Accessibility and Integrity..... 9
2.5.3 File Encoding & Format Support..... 9
2.5.4 Local File System Write Permissions..... 10
2.5.5 Single-User Interactive Execution..... 10
3 Specific Requirements..... 10
3.1 Interface Requirements..... 10
3.1.1 Wireframes..... 10
3.1.1.1 Main Interface..... 10
3.1.1.2 Loading Input Data..... 10
3.1.1.3 Automatic Timetable Generation..... 11
3.1.1.4 Weekly Timetable of a Year Group..... 11
3.1.1.5 Unscheduled Sessions and Warnings..... 11
3.1.1.6 Manual Modification..... 12
3.1.1.7 Classroom Request Email..... 12
3.1.2 Hardware Interfaces..... 12
3.1.3 Software Interfaces..... 12
3.1.4 Communication Interfaces..... 13

3.2 Functional Requirements + Stimulus/Response Sequences..... 13
3.2.1 External Data Ingestion & File Loading..... 13
FR-01: File Parsing Support..... 13
FR-02: Syntactical & Relational Validation..... 13
3.2.2 Automated Timetable Generation Algorithm..... 14
FR-03: Rule-Based Scheduling Execution..... 14
3.2.3 Timetable Presentation & Inspection..... 14
FR-04: View Schedule by Year Group..... 14
FR-05: View Global Schedule & Warnings..... 14
3.2.4 Interactive Assignment Modification..... 15
FR-06: Manual Reassignment & Hard-Constraint Guard..... 15
3.2.5 Unscheduled Session Handling & Official Request Drafting..... 15
FR-07: Classroom Request Email Generation..... 15
3.2.6 Output Export & System Error Handling..... 15
FR-08: Schedule Persistence..... 15
FR-09: Robust Error Interception..... 15
3.3 Use Cases..... 16
UC-01: Load External Data..... 16
UC-02: Run Automatic Timetable Generation..... 16
UC-03: Inspect Timetables & Warnings..... 16
UC-04: Manually Reassign Session..... 16
UC-05: Generate Room Request Letter..... 16
3.4 Non-Functional Requirements..... 17
3.6 Security & Data Integrity..... 17
3.7 Configuration Management..... 17
4 Analysis Models..... 18
4.1 Class Diagram..... 18
4.2 Activity Diagrams..... 19
4.2.1 Generate Automated Timetable..... 19
4.2.2 Activity Diagram: Manual Reassignment & Conflict Guard..... 20
4.2.3 Activity Diagram: Draft Classroom Request Letter..... 21
4.3 Algorithmic Flowcharts..... 22
4.3.2 Automated Timetable Generation Algorithm (FR-03)..... 23
4.3.3 Manual Modification & Hard-Constraint Guard (FR-06)..... 24
4.3 Use Case Diagram..... 25

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document details the complete functional and non-functional requirements for the Department Course Timetable Management System. The system is an automated, standalone Java desktop/console application designed to assist the Department Course Timetable Coordinator in constructing conflict-free weekly schedules for a four-year undergraduate curriculum. The primary objective is to automate the assignment of lectures and laboratory sessions into appropriate weekly time slots and rooms while strictly enforcing all mandatory operational rules (Hard Constraints), respecting academic preferences (Soft Constraints), identifying unresolvable bottlenecks, and facilitating manual modifications.
This document is prepared in accordance with the IEEE Std 830-1998 and ISO/IEC/IEEE Std 29148-2018 standards, serving as the foundational contract between the project development team and academic supervisors for iterative verification and implementation.

1.2 Scope

The scope of this project encompasses designing, developing, verifying, and documenting an object-oriented Java system that:

- Ingests curriculum structures, instructor availabilities, room inventories, and special constraints across multiple heterogeneous external file formats (CSV and TXT/JSON).
- Manages courses distributed over four distinct undergraduate year groups (Years 1 to 4) within the Department of Software Engineering.
- Automatically places theoretical and laboratory teaching blocks into a five-day weekly schedule (Monday through Friday, 09:00–17:00 in one-hour atomic slots) using an algorithmic heuristic.
- Strictly prevents scheduling collisions across instructors, classrooms, laboratory facilities, and student cohorts (year groups).
- Verifies physical room type requirements (Classroom vs. Laboratory) and validates student seating capacities against expected class enrollments.
- Handles fixed-time sessions, special instructor preferences, and sequential requirements (e.g., theory blocks preceding laboratory sessions).
- Flags unassigned sessions and provides an interactive mechanism to generate formal classroom request letters to the Student Affairs Office when internal departmental capacity is exceeded.
- Provides interactive timetable inspection views (per year group and overall) and permits controlled manual reassignments governed by real-time hard-constraint validation.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
CLI	Command-Line Interface; the console-based terminal interface utilized for user interaction.
Room Type	The physical classification of an educational facility: either Classroom (for theoretical lectures) or Laboratory (for practical/technical sessions).
Soft Constraint (SC)	A non-mandatory preference (e.g., theory before lab, instructor preferred hours, minimizing student schedule gaps) that should be satisfied when feasible.
Hard Constraint (HC)	A mandatory scheduling rule that must never be violated under any circumstances (e.g., no instructor double-booking, room capacity limits).
Teaching Block	A continuous, indivisible multi-hour lecture unit (e.g., a 3-hour theory block or a 2-hour lab block) assigned as a single block.
Year Group	A discrete student cohort categorized by academic progression (Year 1, Year 2, Year 3, Year 4). Courses within the same cohort cannot overlap.
Coordinator	Department Course Timetable Coordinator; the primary human operator configuring and generating schedules.
SDD	Software Design Document; architectural and detailed object design specification.
SRS	Software Requirements Specification; document specifying system requirements and behavior.

1.4 References

This section lists the references used in the development of this Software Requirements Specification (SRS) document for the **Timetable Management System** project:

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Computer Society, 1998.
- ISO/IEC/IEEE Std 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE, 2018.
- Course Specification and Assignment Guideline: *SENG 271 Software Project Assignment – Department Course Timetable Management System*, Çankaya University, Department of Software Engineering.
- Oracle Corporation, *Java Standard Edition (Java SE 17/21) Language and Library Specifications*, 2025.
- Gamma, E., Helm, R., Johnson, R., Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

1.5 Overview

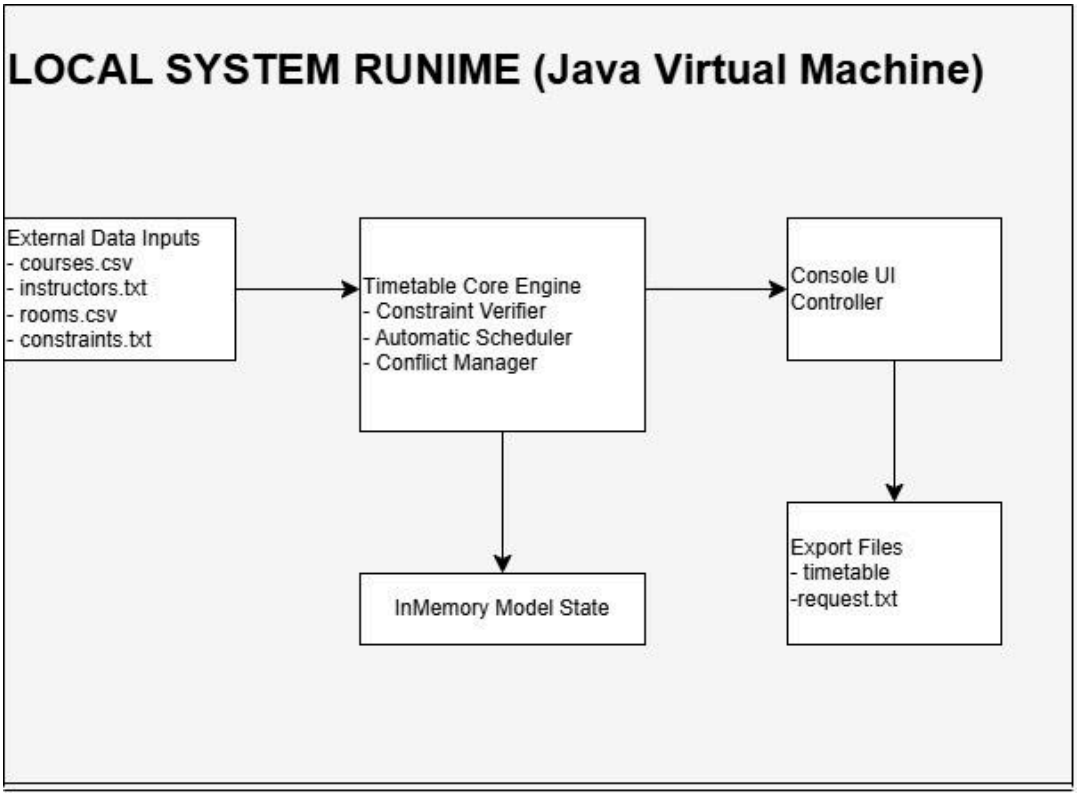
- **Section 2:** Provides a general overview of the product perspective, architectural context, functional decomposition, user characteristics, general operating constraints, and system assumptions/dependencies.
- **Section 3:** Outlines external interface requirements (CLI, hardware, software, files), detailed functional requirements with Stimulus/Response Sequences, Use Case models, non-functional requirements (performance, reliability, maintainability), design constraints, and data integrity considerations.
- **Section 4:** Presents structural and dynamic analysis models including Class Specifications, Use Case Diagrams, Activity Flows, and Algorithm Execution Flowcharts.

2 General Description

2.1 Product Perspective

The Department Course Timetable Management System is an independent, self-contained desktop software package developed in Java. It operates locally on the coordinator's workstation without external server, cloud, or network dependencies.

The software acts as a decision-support and schedule-generation engine. It ingests flat-file input datasets defining the curriculum, faculties, and facilities, applies a multi-constraint rule engine to produce a viable schedule, enables manual inspection and adjustments, and outputs structured timetables along with administrative notification drafts.



2.2 Product Functions

The **Seng271** platform provides the following major functionalities:

2.2.1 Data Management & Parsing

- Parses external curriculum, instructor, classroom, and constraint configuration files.
- Supports heterogeneous formats (CSV and TXT/JSON).
- Validates data completeness, data types, and cross-entity references.

2.2.2 Automatic Timetable Generation

- Categorizes sessions into theoretical (-T) and laboratory (-L) continuous teaching blocks.
- Prioritizes fixed-time courses and mandatory constraints.
- Performs sequential slot-searching across the academic week (Mon–Fri, 09:00–17:00).
- Applies all Hard Constraints (HC-01 to HC-09) and evaluates Soft Constraints (SC-01 to SC-04).
- Isolates and flags unschedulable sessions with detailed failure diagnostics.

2.2.3 Timetable Viewing & Presentation

- Displays complete department-wide scheduled sessions.
- Generates formatted weekly calendar grids partitioned by Year Group (Years 1, 2, 3, and 4).
- Provides comprehensive summaries highlighting scheduled totals, unscheduled sessions, and soft-constraint warning notices.

2.2.4 Manual Modification & Conflict Validation

- Enables the coordinator to manually shift, reassign, or clear scheduled course blocks.
- Executes real-time hard-constraint checks against proposed modifications, rejecting conflicting changes.
- Issues explicit warnings for soft-constraint violations while permitting coordinator discretion.

2.2.5 Unscheduled Session & Classroom Request Management

- Categorizes unscheduled sessions by root cause (NO ROOM vs. CONSTRAINT CONFLICT).
- Generates formal, standardized email drafts to the Student Affairs Office requesting additional classrooms/labs for courses that failed due to internal capacity shortages.

2.2.6 Export & Output Persistence

- Exports generated weekly schedules into formatted plain text (.txt) and comma-separated (.csv) files.

- Persists formal classroom request letters to local output directories (output/classroom_request_<CourseCode>.txt).

2.3 User Characteristics

2.3.1 Department Course Timetable Coordinator

- The primary operator of the system.
- Possesses full knowledge of departmental curriculum structures, academic priorities, and faculty teaching preferences.
- Has standard computer literacy (navigating command-line menus, editing plain text/CSV files, inspecting reports). Does not require software programming knowledge to operate the compiled tool.

2.4 General Constraints

2.4.1 Programming Language & Platform Environment

- The core application must be written entirely in standard Java (Java SE 17 or higher) using core object-oriented features.
- Must run cross-platform on any standard operating system hosting a compatible JVM (Linux, Windows, macOS).

2.4.2 Execution Paradigm & User Interface Constraints

- Primary interaction is governed via an interactive Console Menu (Command-Line Interface). A graphical user interface (JavaFX or Swing) is completely optional.

2.4.3 Algorithmic & Architectural Scope Limits

- No external optimization engines or third-party file parsing frameworks must be used.
- Complex backtracking or meta-heuristics are out of scope; a deterministic, rule-based algorithmic placement approach is sufficient.

2.4.4 Scheduling & Academic Boundary Constraints

- The academic week is strictly limited to 5 operating days: Monday through Friday.
- Daily operational hours are restricted to 09:00 to 17:00 (8 atomic one-hour slots per day: 09–10, 10–11, 11–12, 12–13, 13–14, 14–15, 15–16, 16–17).
- Multi-hour sessions must be scheduled as consecutive, uninterrupted blocks; sessions cannot span across day boundaries or exceed the 17:00 threshold.

2.4.5 Hard Constraint Inviolability

The algorithm and manual editing workflows must strictly enforce the following Hard Constraints:

- HC-01 (Instructor Conflict): An instructor cannot teach multiple sessions simultaneously.
- HC-02 (Room Conflict): A physical room cannot host multiple sessions concurrently.

- HC-03 (Year-Group Conflict): Mandatory courses for the same year group cannot overlap.
- HC-04 (Working Hours): Sessions must occur only between 09:00 and 17:00, Monday through Friday.
- HC-05 (Room Capacity): Room capacity must be greater than or equal to estimated class enrollment.
- HC-06 (Room Type): Theoretical sessions require Classroom; practical sessions require Laboratory.
- HC-07 (Instructor Availability): Sessions cannot be scheduled during an instructor's declared unavailable hours.
- HC-08 (Fixed-Time Requirements): Mandatory pre-set day/time requests must be respected.
- HC-09 (Room Availability): Sessions cannot be assigned to rooms during room maintenance/unavailable slots.

2.4.6 Soft Constraint Flexibility & Warning Notification

Soft constraints represent preferences. Violations do not invalidate a schedule but trigger system warnings:

- SC-01 (Theory Before Laboratory): A course's lab block should be scheduled after its lecture block.
- SC-02 (Instructor Preferences): Preferred time slots (e.g., mornings) should be honored where feasible.
- SC-03 (Avoiding Unnecessary Gaps): Idle multi-hour gaps in student year-group daily schedules should be minimized.
- SC-04 (Consecutive Teaching Hours): Teaching hours within a block must remain consecutive.

2.4.7 Special Scheduling Requests

Request type	Example	Mandatory or preferred	How the system handles it
Fixed time	SENG301 must be on Wednesday, 13:00–16:00	[ ]	[ ]
Specific room	SENG401 must use Laboratory B202	[ ]	[ ]
Instructor unavailable day	SENG201 instructor unavailable on Friday	[ ]	[ ]
Laboratory before theory	SENG302 laboratory may precede theory	[ ]	[ ]
[Other]			

2.5 Assumptions and Dependencies

The correct operation of the **Timetable Management System** platform depends on the following assumptions and external dependencies:

2.5.1 Runtime & Execution Dependencies

- The target system must have the Java Development Kit (JDK 17+) or Java Runtime Environment (JRE) installed.

2.5.2 Input File Accessibility and Integrity

- Input data files (courses.csv, instructors.txt, rooms.csv, constraints.txt) must reside in the expected project directory path (e.g., ./data/).

2.5.3 File Encoding & Format Support

- Files are assumed to be plain text encoded in UTF-8 or ASCII with standard newline delimiters (\n or \r\n).

2.5.4 Local File System Write Permissions

- The runtime user must have operating-system write permissions to create and update files within the ./output/ directory.

2.5.5 Single-User Interactive Execution

- The application is assumed to operate synchronously as a single-user CLI process; concurrent multi-threaded modifications are not required

3 Specific Requirements

3.1 Interface Requirements

3.1.1 Wireframes

3.1.1.1 Main Interface

```
==================================================
   DEPARTMENT COURSE TIMETABLE MANAGEMENT SYSTEM
          Software Engineering Department
==================================================
 1. Load input data
 2. Generate timetable automatically
 3. View timetable by year group
 4. View all scheduled sessions
 5. View unscheduled sessions and warnings
 6. Modify / remove an assignment
 7. Generate classroom request email
 8. Save timetable to file
 0. Exit
--------------------------------------------------
Select an option: 1
```

3.1.1.2 Loading Input Data

```
=== LOAD INPUT DATA ===
Reading data/courses.csv ........ 6 courses loaded
Reading data/instructors.txt .... 5 instructors loaded
Reading data/rooms.csv .......... 4 rooms loaded
Reading data/constraints.txt .... 3 requests loaded (2 mandatory, 1 preferred)

WARNINGS:
  courses.csv, line 8: invalid value "three" for theory hours -> record skipped
  constraints.txt, line 5: course "SENG499" not found -> request ignored

Data loaded with 2 warnings.
Press Enter to return to the main menu...
```

3.1.1.3 Automatic Timetable Generation

```
=== AUTOMATIC TIMETABLE GENERATION ===
Step 1/2: Processing fixed-time requests...
  [OK]   SENG301-T  Wed 13:00-16:00  A102   (fixed by request)
  [FAIL] SENG305-T  Fixed time Fri 09:00-12:00 violates HC-07:
                    instructor Dr. Sahin is unavailable on Friday

Step 2/2: Scheduling remaining sessions...
  [OK]   SENG101-T  Mon 09:00-12:00  A101
  [FAIL] SENG101-L  HC-05: no laboratory with capacity >= 70
  [OK]   SENG201-T  Tue 09:00-12:00  A101   (after 4 rejected attempts)
  [FAIL] SENG201-L  HC-05: no laboratory with capacity >= 55
  [OK]   SENG203-T  Mon 09:00-12:00  A102
  [OK]   SENG203-L  Wed 13:00-15:00  B202
  [OK]   SENG401-T  Mon 13:00-15:00  A102
  [OK]   SENG401-L  Tue 13:00-15:00  B202   (special request: must use B202)
  [WARN] SENG401-T  SC-02: Dr. Aksoy prefers mornings; placed at 13:00

------------------------------------------------
SUMMARY  Total sessions: 10 | Scheduled: 7 | Unscheduled: 3 | Warnings: 1
------------------------------------------------
```

3.1.1.4 Weekly Timetable of a Year Group

```
=== WEEKLY TIMETABLE — YEAR 2 ===
+-------------+---------------+---------------+---------------+----------+--------
--+
| Time        | Monday        | Tuesday       | Wednesday     | Thursday | Friday
|
+-------------+---------------+---------------+---------------+----------+--------
| 09:00-10:00 | SENG203-T A102 | SENG201-T A101 |               |          |
|
| 10:00-11:00 | SENG203-T A102 | SENG201-T A101 |               |          |
|
| 11:00-12:00 | SENG203-T A102 | SENG201-T A101 |               |          |
|
| 12:00-13:00 |               |               |               |          |
|
| 13:00-14:00 |               |               | SENG203-L B202 |          |
|
| 14:00-15:00 |               |               | SENG203-L B202 |          |
|
| 15:00-16:00 |               |               |               |          |
|
| 16:00-17:00 |               |               |               |          |
|
+-------------+---------------+---------------+---------------+----------+--------
--+
T = Theory, L = Laboratory

Session details:
  SENG201-T  Object-Oriented Programming  Dr. Demir  Tue 09:00-12:00  A101
  SENG203-T  Data Structures              Dr. Kaya   Mon 09:00-12:00  A102
  SENG203-L  Data Structures              Dr. Kaya   Wed 13:00-15:00  B202

Not scheduled for Year 2: SENG201-L (2 h) -> see option 5
```

3.1.1.5 Unscheduled Sessions and Warnings

```
=== UNSCHEDULED SESSIONS ===
#  Session    Year  Hours  Students  Problem type   Reason
1  SENG101-L  1     2      70        NO ROOM        No laboratory with capacity >= 70
2  SENG201-L  2     2      55        NO ROOM        No laboratory with capacity >= 55
3  SENG305-T  3     3      40        CONSTRAINT     Fixed time (Fri) conflicts with
                                                   instructor unavailability (HC-07)

-> Sessions 1-2 can be solved with a classroom request (option 7).
-> Session 3 must be resolved by the coordinator (change the request or contact the
instructor).

=== SOFT-CONSTRAINT WARNINGS ===
- SC-02  SENG401-T placed on Mon 13:00; Dr. Aksoy prefers morning hours.
```

3.1.1.6 Manual Modification

```
=== MODIFY AN ASSIGNMENT ===
Session to modify: SENG203-L
Current assignment: Wed 13:00-15:00, B202

New day (Mon-Fri): Tue
New start time   : 13:00
New room         : B202

Checking hard constraints...
  [X] HC-02 Room conflict: B202 is used by SENG401-L on Tue 13:00-15:00
Modification REJECTED. The current assignment is kept.

Try another option? (y/n): y
New day (Mon-Fri): Mon
New start time   : 15:00
New room         : B202

Checking hard constraints...
  [OK] All hard constraints satisfied.
  [!]  SC-03 Year 2 will have a 3-hour gap on Monday (12:00-15:00).
Apply the change anyway? (y/n): y

Assignment updated: SENG203-L -> Mon 15:00-17:00, B202
```

3.1.1.7 Classroom Request Email

```
=== CLASSROOM REQUEST ===
Sessions without a suitable room:
  1. SENG101-L  (Year 1, 70 students, Laboratory, 2 h)
  2. SENG201-L  (Year 2, 55 students, Laboratory, 2 h)
Select a session: 2
Suggested time (year group and instructor both free): Tuesday 13:00-15:00

------------------- EMAIL DRAFT -------------------
Subject: Classroom Request – SENG201

Dear Student Affairs Office,

We are preparing the department course timetable for the upcoming semester.
A suitable classroom could not be assigned for the following session:

Course: SENG201 – Object-Oriented Programming (Laboratory)
Number of Students: 55
Required Room Type: Computer Laboratory
Preferred Time: Tuesday, 13:00–15:00

We kindly request your assistance in identifying an available room
that meets these requirements.

Best regards,
Department Timetable Coordinator
---------------------------------------------------
Save this email to a file? (y/n): y
Saved to output/classroom_request_SENG201.txt
```

3.1.2 Hardware Interfaces

- Standard personal computer terminal displaying at least 80x24 characters.
- Standard keyboard input for alphanumeric and menu selections.

3.1.3 Software Interfaces

- Java Virtual Machine (JVM): Relies on standard Java runtime APIs (java.lang, java.util, java.io).
- File System Interface: Interacts with local storage for reading configuration data (/data) and writing scheduling logs (/output).

3.1.4 Communication Interfaces

- No direct network or TCP/IP socket communication is used. All data transfer occurs via local file input/output streams.

3.2 Functional Requirements + Stimulus/Response Sequences

3.2.1 External Data Ingestion & File Loading

FR-01: File Parsing Support

- Description: The system shall parse input datasets from at least two distinct formats: Comma-Separated Values (.csv) and Plain Formatted Text (.txt).
- Inputs: courses.csv, instructors.txt, rooms.csv, constraints.txt.

FR-02: Syntactical & Relational Validation

- Stimulus: Coordinator selects Option 1 from the main menu.
- Response: System iterates through each record in each file. If an invalid data type is encountered (e.g., text instead of integer hours) or an unreferenced foreign key appears (e.g., an unknown instructor ID), the system skips or flags the invalid record, prints a diagnostic warning, loads valid records, and returns to the menu without crashing.

3.2.2 Automated Timetable Generation Algorithm

FR-03: Rule-Based Scheduling Execution

- Stimulus: Coordinator selects Option 2 from the main menu.
- Sequence:
    1. System identifies courses with fixed-time constraints (HC-08) and schedules them first, validating room and instructor availability.
    2. System extracts unscheduled courses and splits each into continuous teaching blocks: Theoretical (-T) and Laboratory (-L).
    3. For each block, the algorithm iterates systematically through operational days (Mon–Fri) and valid consecutive time slots.
    4. Evaluates compatible rooms matching room type (HC-06) and room capacity (HC-05).
    5. Checks instructor availability (HC-01, HC-07) and year-group concurrency (HC-03).
    6. If a candidate slot satisfies all HCs, the session is assigned. If soft constraints are breached (e.g., SC-02 instructor preferred time), a warning is logged.
    7. If all combinations violate one or more HCs, the block is registered into the Unscheduled Sessions registry.
    8. System displays execution completion metrics (Total, Scheduled, Unscheduled, Warnings).

3.2.3 Timetable Presentation & Inspection

FR-04: View Schedule by Year Group

- Stimulus: Coordinator selects Option 3 and inputs target year group (1, 2, 3, or 4).
- Response: System renders a 5-day by 8-hour grid showing course codes, session types, and assigned room identifiers, followed by a list of unscheduled courses for that year group.

FR-05: View Global Schedule & Warnings

- Stimulus: Coordinator selects Option 4 or Option 5.
- Response: System displays all scheduled sessions department-wide or outputs a categorized diagnostic report segregating room shortage issues from constraint conflicts.

3.2.4 Interactive Assignment Modification

FR-06: Manual Reassignment & Hard-Constraint Guard

- Stimulus: Coordinator selects Option 6, identifies a session (e.g., SENG203-L), and inputs proposed Day, Time, and Room.
- Response:
    - If the change violates any Hard Constraint (HC-01 through HC-09), the system prints [X] Modification REJECTED, explains the conflict, preserves the original state, and prompts for alternative input.
    - If all Hard Constraints pass, the system checks for Soft Constraints (e.g., creating a 3-hour student gap SC-03). It prompts: Apply the change anyway? (y/n). Upon confirmation (y), the assignment is updated.

3.2.5 Unscheduled Session Handling & Official Request Drafting

FR-07: Classroom Request Email Generation

- Stimulus: Coordinator selects Option 7, views sessions flagged with NO ROOM, and chooses a target course.
- Response: The system determines the required room type, student count, and a conflict-free suggested time slot (where instructor and year group are free). It formats a formal email draft addressed to the Student Affairs Office and provides an option to save the text to output/classroom_request_<CourseCode>.txt.

3.2.6 Output Export & System Error Handling

FR-08: Schedule Persistence

- Stimulus: Coordinator selects Option 8.
- Response: System serializes the current schedule into human-readable text files and structured CSV spreadsheets in the output/ directory.

FR-09: Robust Error Interception

- Description: All user console inputs (non-numeric menu choices, out-of-bounds slot selections) and file I/O operations (missing source files) must be handled gracefully with descriptive error prompts without terminating the application process.

3.3 Use Cases

UC-01: Load External Data

- Actor: Coordinator.
- Precondition: Input files placed in ./data/.
- Main Flow: Coordinator triggers data load; system validates records, populates domain models, and logs parsing warnings.

UC-02: Run Automatic Timetable Generation

- Actor: Coordinator.
- Precondition: Data loaded successfully.
- Main Flow: System executes two-stage scheduling algorithm (fixed sessions first, followed by flexible sessions), testing HC-01 through HC-09 and SC-01 through SC-04.

UC-03: Inspect Timetables & Warnings

- Actor: Coordinator.
- Main Flow: Coordinator requests timetable output per cohort or diagnostic summary of unassigned sessions.

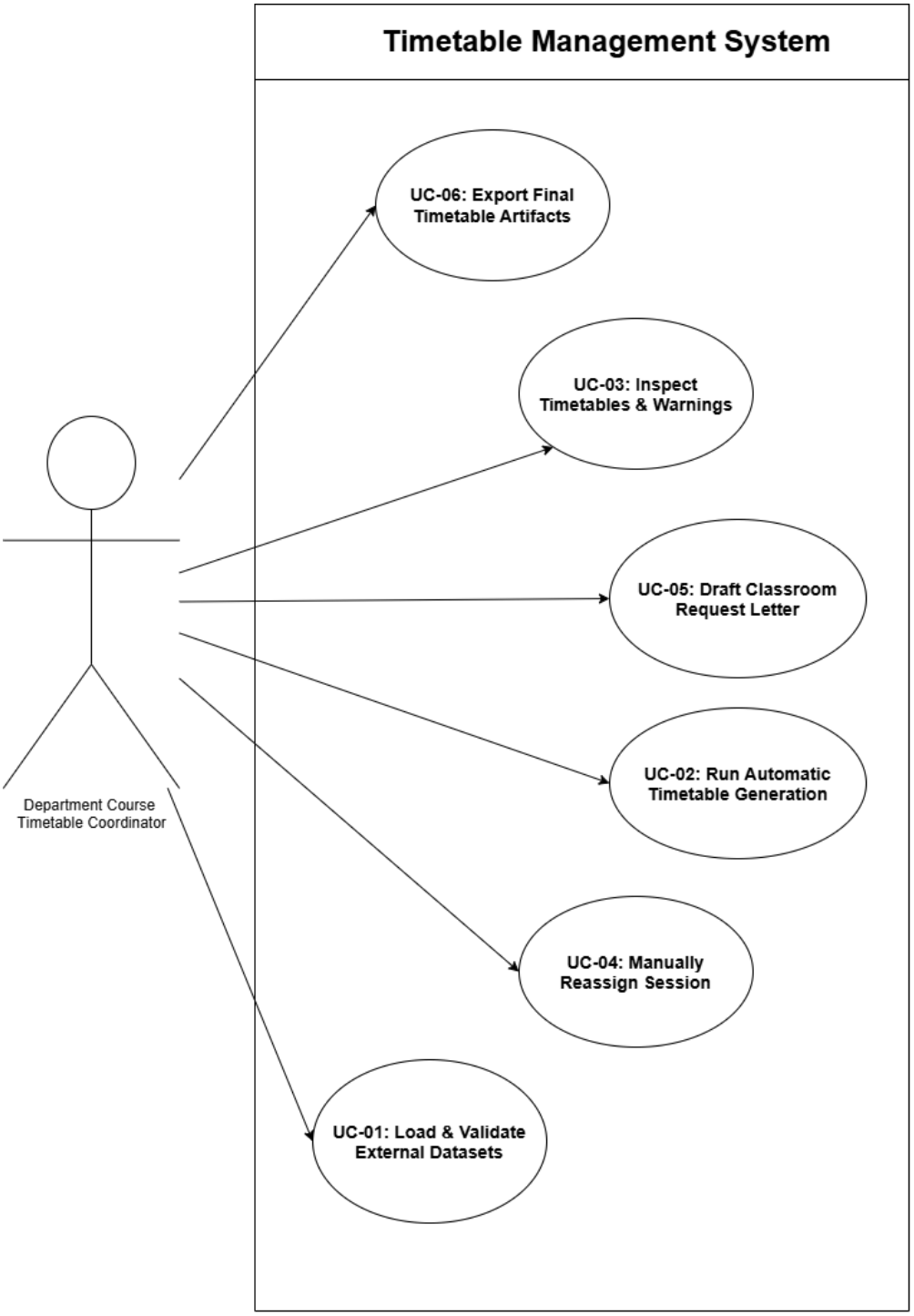
UC-04: Manually Reassign Session

- Actor: Coordinator.
- Main Flow: Coordinator selects session and provides target slot/room. System validates hard constraints. If valid, updates schedule; if invalid, reverts.

UC-05: Generate Room Request Letter

- Actor: Coordinator.
- Precondition: Unscheduled sessions exist due to room capacity or room type shortfalls.
- Main Flow: System drafts formal email text with required parameters and saves it to a file.

3.3.1 Use Case Diagram



3.4 Non-Functional Requirements

- NFR-01 (Performance & Efficiency): Automated schedule generation for a standard departmental load (approx. 20–50 courses across 4 year groups) must execute and complete within 5 seconds on standard desktop hardware.
- NFR-02 (Reliability & Fault Tolerance): Faulty records, missing files, or corrupted rows must not crash the application. The system must log warnings and continue processing valid records.
- NFR-03 (Usability & Readability): Timetable outputs rendered in console and text files must be structured into clear ASCII grids. Diagnostic errors must clearly indicate the constraint code (e.g., HC-02, HC-05, SC-02).
- NFR-04 (Portability): The software must compile and execute on any platform hosting Java SE 17+ without modifications or native platform dependencies.
- NFR-05 (Maintainability): Code must follow object-oriented design principles (SOLID), featuring clear separation of concerns among models, parsers, scheduling logic, and user interfaces.

3.5 Design Constraints

- Language & Runtime: Pure Java implementation without external scheduling frameworks.
- Indivisible Teaching Blocks: A course's theoretical block (e.g., 3 hours) and lab block (e.g., 2 hours) cannot be broken into arbitrary 1-hour fragments across different days.
- Architecture: Class responsibilities must align cleanly with the problem domain: Course, Instructor, Room, TimeSlot, Timetable, ConstraintChecker, Scheduler, and Main.

3.6 Security & Data Integrity

- Local Execution Security: The application requires only local user permissions. It executes within standard user-space privileges without requiring administrative or root elevation.
- Input Data Sanitization: Input values (numeric capacities, hour values, string identifiers) must be validated and sanitized before domain object instantiation.
- Schedule Consistency: The internal state of the Timetable object must maintain referential integrity. If an assignment is rejected during manual modification, the previous valid state must remain unchanged.

3.7 Configuration Management

- Source code versioned using GitHub.
- Changes to critical configurations must be documented and version-controlled.

3.8 Input Data Requirements

File	Format	Fields (in order)	Example line	Validation rules
`data/courses.csv`	CSV	CourseCode, Title, YearGroup, InstructorID, TheoryHours, LabHours, Enrollment, RoomType	SENG201,OOP,2,INST01,3,2,55,Classroom	YearGroup is 1–4; hours and enrollment are positive integers; InstructorID must exist in instructors.txt.
`data/instructors.txt`	TXT	InstructorID, FullName, UnavailableDays/Hours, PreferredTimes	INST01,Dr. Demir,Fri 09:00-17:00,Morning	[InstructorID must be unique; time formats must adhere to HH:00-HH:00 syntax.]
`data/rooms.csv`	CSV	RoomID, Building, Capacity, RoomType	A101,Main,60,Classroom	RoomID must be unique; Capacity must be an integer > 0; RoomType must be Classroom or Laboratory.
`data/constraints.txt`	TXT	CourseCode, ConstraintType, Value, Priority	SENG301,FixedTime,Wed 13:00-16:00,Mandatory	CourseCode must exist in courses.csv; Priority must be Mandatory or Preferred.

3.8.1 Input Error Handling

Error	Example	System reaction
Missing file	`rooms.csv` not found	System catches FileNotFoundException, displays a clear error message, and safely halts data loading without crashing.
Invalid numeric value	"three" for theory hours	System skips the invalid record, logs: courses.csv, line 8: invalid value "three" -> record skipped, and continues loading remaining records.
Incomplete record	A course line with a missing field	System detects column count mismatch, logs a line-level parsing error, and skips the malformed row.
Reference to a nonexistent item	A request for course SENG499, which is not in `courses.csv`	System logs: constraints.txt, line 5: course "SENG499" not found -> request ignored, and discards the invalid rule.

3.9 Output Data Requirements

Output	Content	Shown on screen	Saved to file
Weekly timetable per year group	Day, time, course code, course title, instructor, room in a structured 5-day ASCII grid.	Yes (Screen 4)	output/timetable_year_<N>.txt
All scheduled sessions	Complete department-wide schedule listing day, slot, course, instructor, room, and cohort.	Yes (Option 4)	output/complete_timetable.csv / .txt
Unscheduled sessions	Session code, year group, hours, student count, problem type (NO ROOM / CONSTRAINT), failure explanation.	Yes (Screen 5)	Included in summary report output/timetable_summary.txt
Warnings	Soft-constraint notices (SC-01 to SC-04) and file parsing diagnostics.	Yes (Screen 2, Screen 3, Screen 5)	Included in summary log output/timetable_summary.txt
Classroom request email draft	Formal letter text containing course code, enrollment, required room type, and suggested conflict-free slot.	Yes (Screen 7)	output/classroom_request_<CourseCode>.txt

8. Changes from the Previous Version

ID or section	Change (added / changed / removed)	Reason for the change
All sections	Added (Initial Version v1)	Initial baseline specification created for submission.

AI-Use Declaration

Tool	Used for	How we checked the result
Gemini / LLM Assistant	Restructuring existing SRS technical content into the standard template format and drafting requirement sentences	Manually verified against project specifications, checked all constraint IDs (HC/SC), confirmed wireframe screen mappings, and validated input/output table schemas.